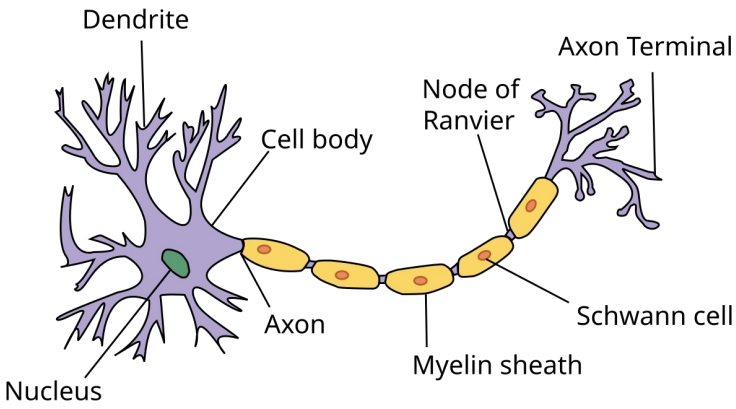
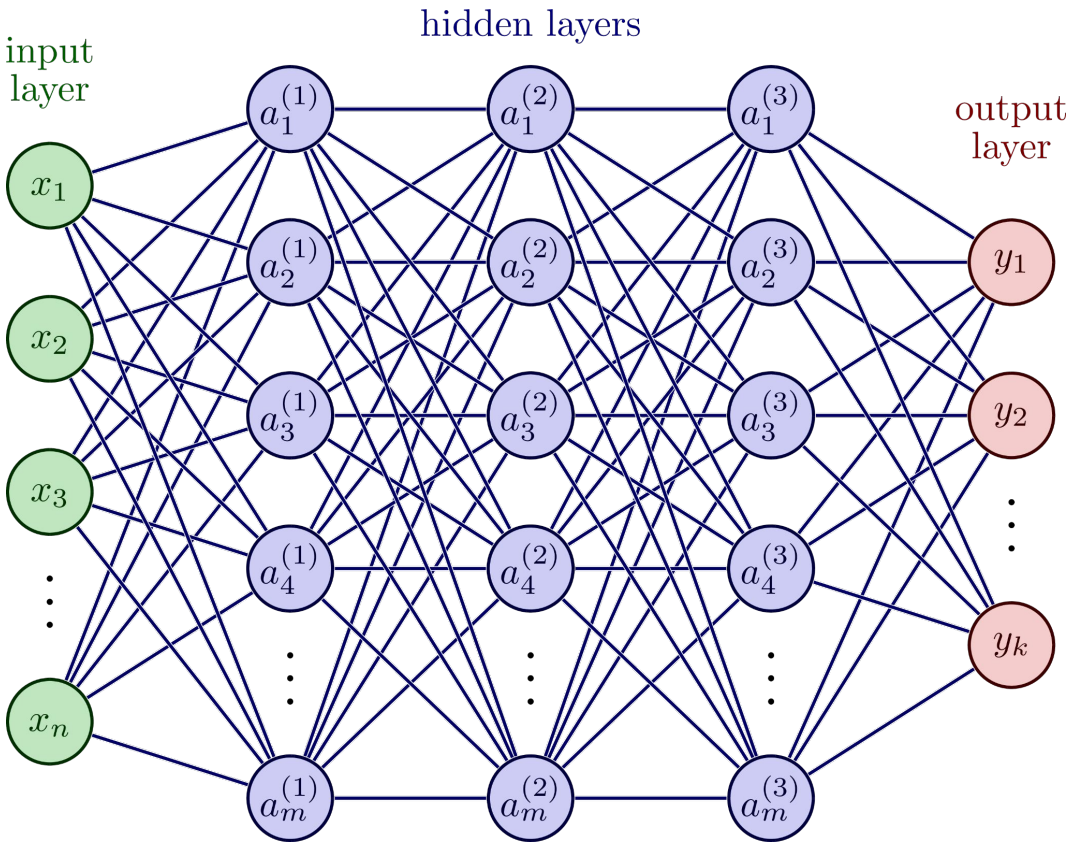
Neural Networks

A language model need not be a neural network

- Though all of today's LLMs are neural models, you can model language without neural networks
- N-gram models and Hidden Markov Models (HMMs) are examples of non-neural approaches to language modeling
- So why use neural networks?


Dendrite
Cell body
Axon
Nucleus
Node of Ranvier
Axon Terminal
Schwann cell
Myelin sheath


input layer
hidden layers
output layer
x_1
x_2
x_3
x_n
$a_1^{(1)}$ $a_2^{(1)}$ $a_3^{(1)}$ $a_4^{(1)}$ $a_m^{(1)}$
$a_1^{(2)}$ $a_2^{(2)}$ $a_3^{(2)}$ $a_4^{(2)}$ $a_m^{(2)}$
$a_1^{(3)}$ $a_2^{(3)}$ $a_3^{(3)}$ $a_4^{(3)}$ $a_m^{(3)}$
y_1
y_2
y_k

First, regression

- Before going over neural networks, let's revisit regression
- Multiple linear regression:

$$y_i = \beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}$$

- Using features 1…p in observation i, we want to estimate y_i.

First, regression

- In logistic regression, we use the sigmoid function to turn this linear model into something that lets us estimate probabilities:

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

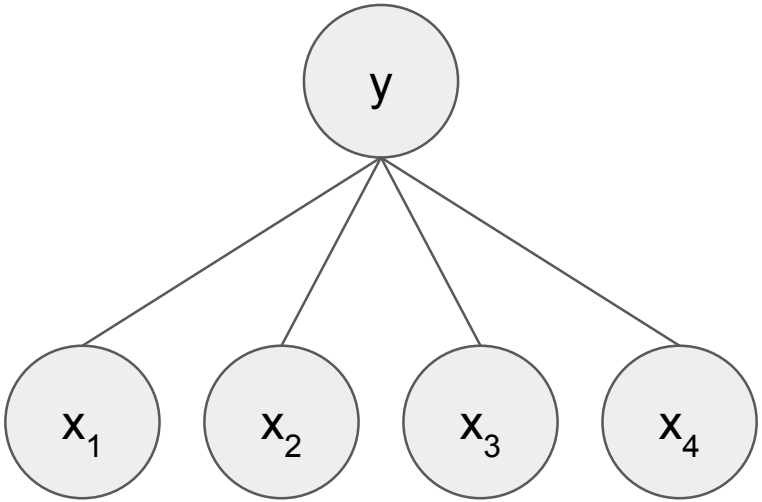
$$t_i = \beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}$$

- The sigmoid function is *nonlinear* and has outputs between 0 and 1.

So what does this have to do with NNs?

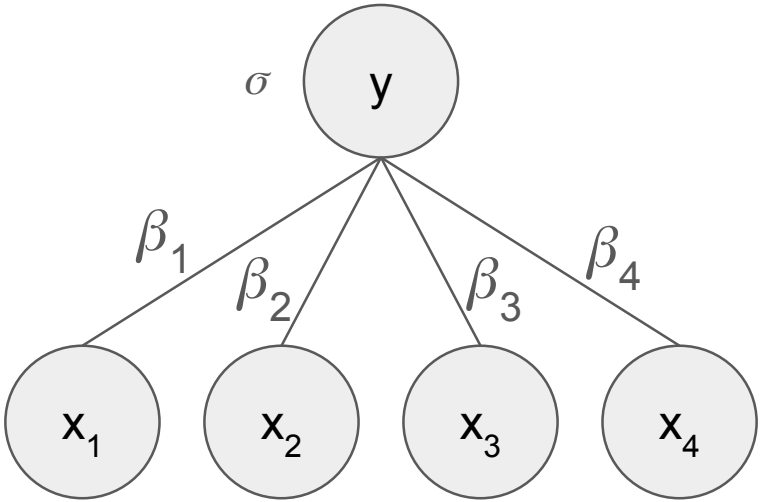
So what does this have to do with NNs?

- A logistic regression model *is* a neural network, albeit a very simple one:

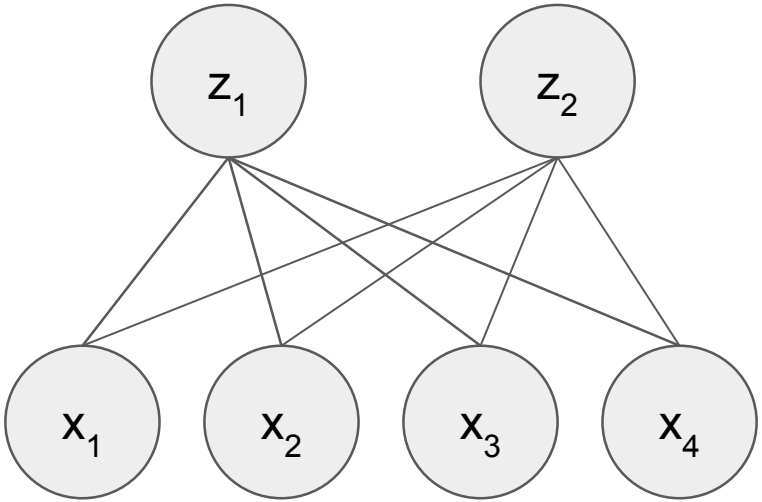


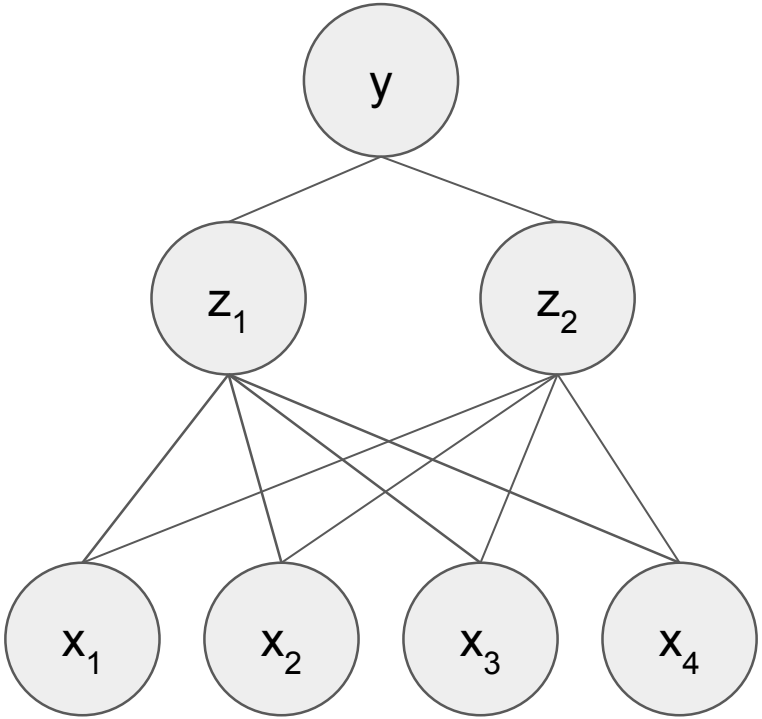
Logistic regression is a neural net

- Data flows from bottom-to-top: circles are inputs/outputs, lines represent multiplication

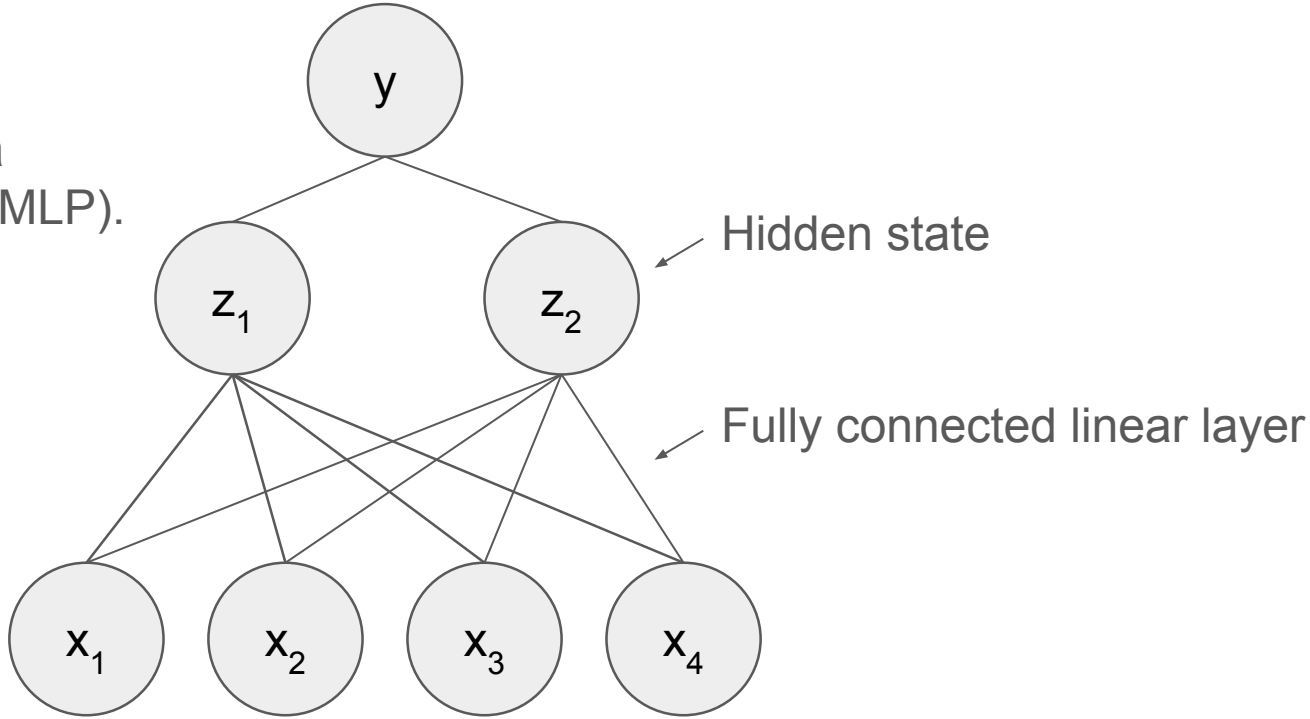


- But we can do more than predict a single value!




y
z_1
z_2
x_1
x_2
x_3
x_4

We call this structure a *multilayer perceptron* (MLP).



Some vocab

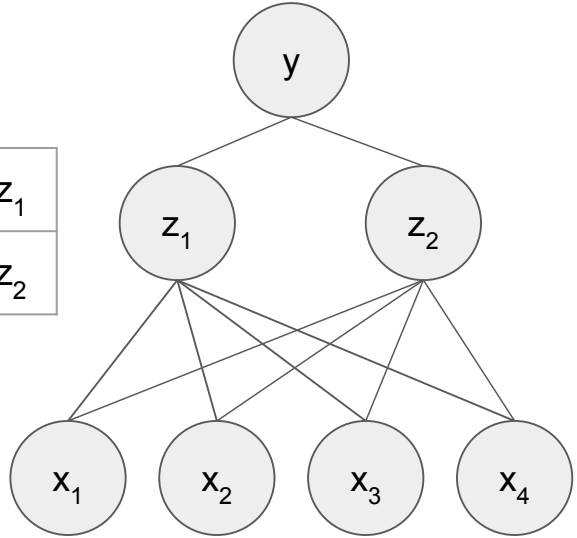
- **Weights/Parameters**: roughly equivalent to coefficients in a regression model. Learned during training.
- **Feed-forward**: Input is multiplied by weights, giving us output.
- **Layer**: A parameterized function that maps input to output.
- **Activation function**: Like sigmoid in logistic regression, a function that adds nonlinearity so the network can learn more complex functions.
- **Deep**: Having multiple layers.
- **Loss function**: A function that maps (prediction, label) pairs to a value (the loss) that measures how wrong the model's predictions are

Why do we use neural networks?

- Universal approximation theorem
  - Neural networks with a certain structure can approximate any continuous function.
  - In other words: if you have the right NN structure, you can learn pretty much any input/output mapping (in theory).
- Practically speaking, neural networks are very well supported by a mature development ecosystem
  - This is obvious to say now, but was not always the case
- Today's successful architectures are highly parallelizable using GPUs
  - Why is this?

Forward pass as matrix multiplication

$$\sigma\left(\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \end{bmatrix} @ \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + a_{14}x_4 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + a_{24}x_4 \end{bmatrix}\right) = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$$



In language models

- In an LLM, the last model layer is just a single, fully-connected linear layer that takes a dense representation generated by all the other layers of the model and outputs a prediction over the vocabulary.
  - (L_sequence, D) -> (L_sequence, V)
- There are MLPs in the transformer architecture elsewhere as well
- Attention (which we'll talk about later) is also a neural architecture

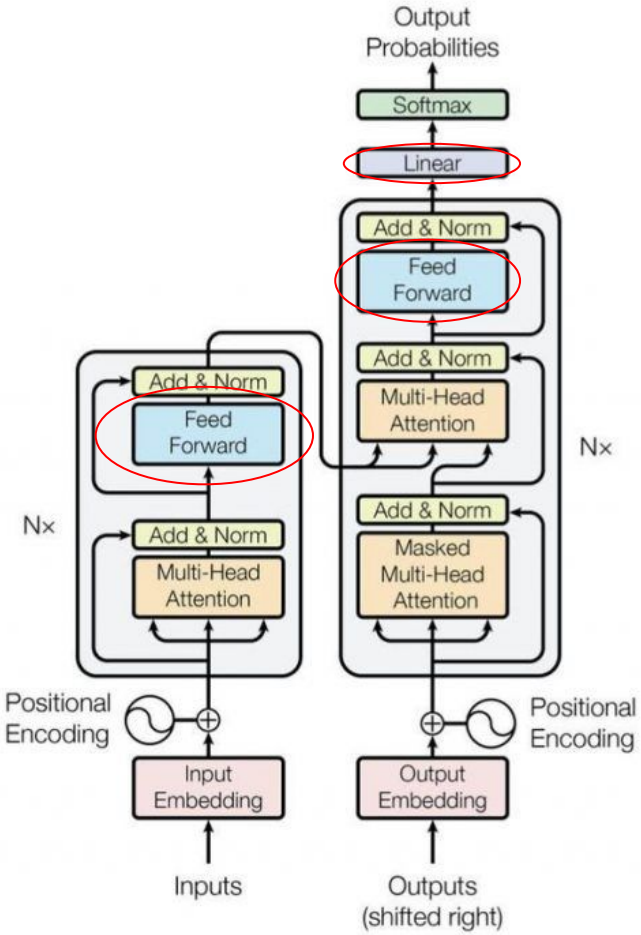


Figure 1: The Transformer - model architecture.

Visualizing matrix multiplication

https://pytorch.org/blog/inside-the-matrix/